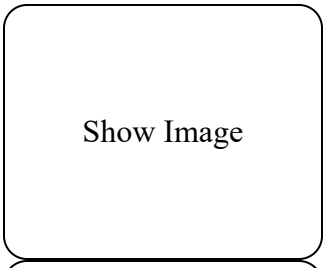
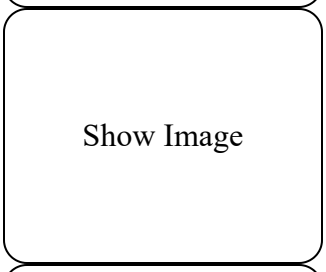
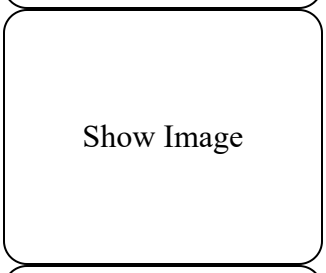
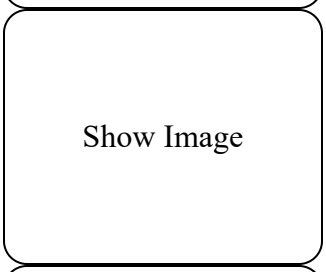
Credit Card Customer Churn Prediction Pipeline

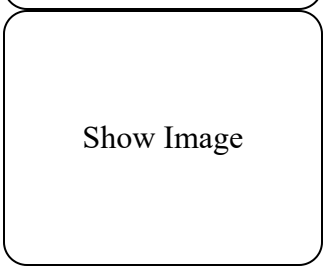
End-to-End ML Solution on AWS using Apache Spark











📋 Table of Contents

🎯 Project Overview

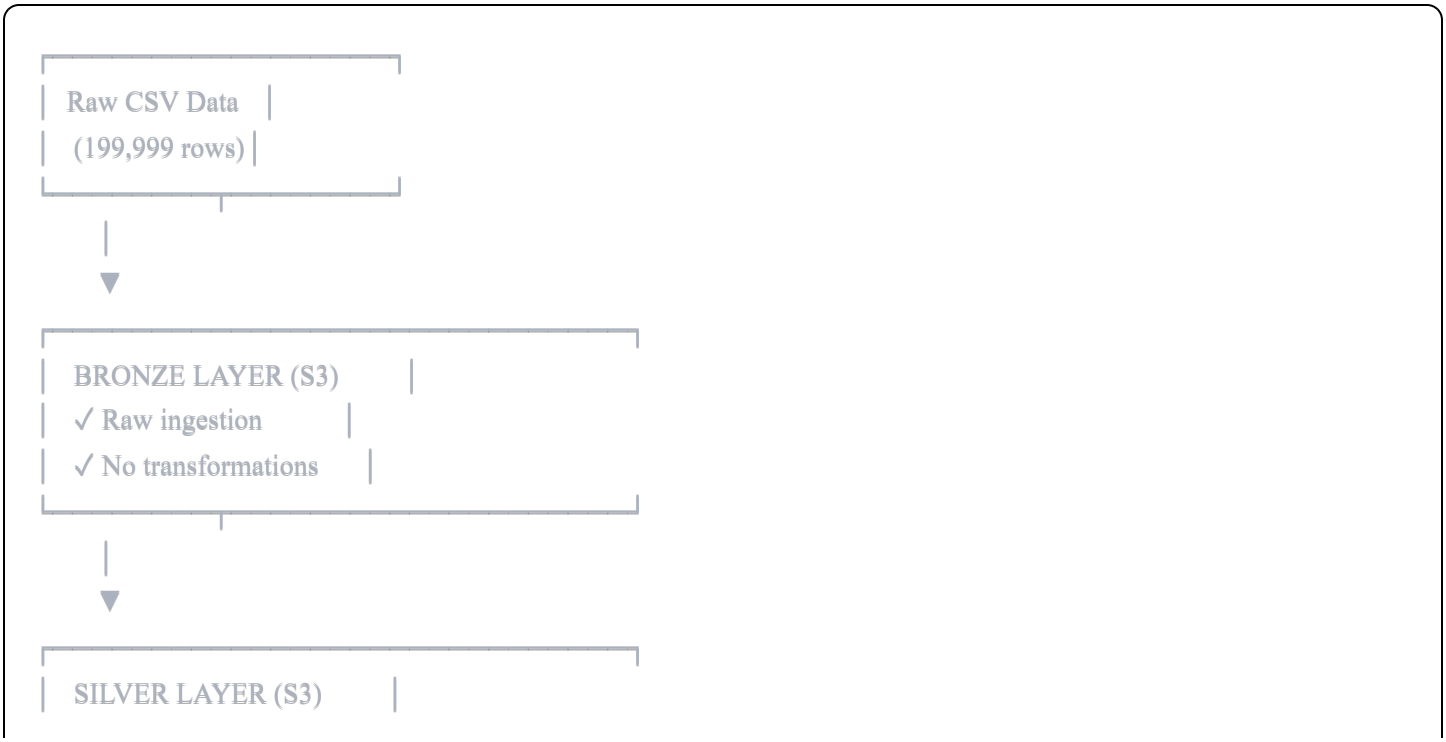
Built a production-grade machine learning pipeline to predict credit card customer churn using the **Medallion Architecture** on AWS. The pipeline processes 200K+ customer records, achieving **80% AUC-ROC** with Random Forest classification.

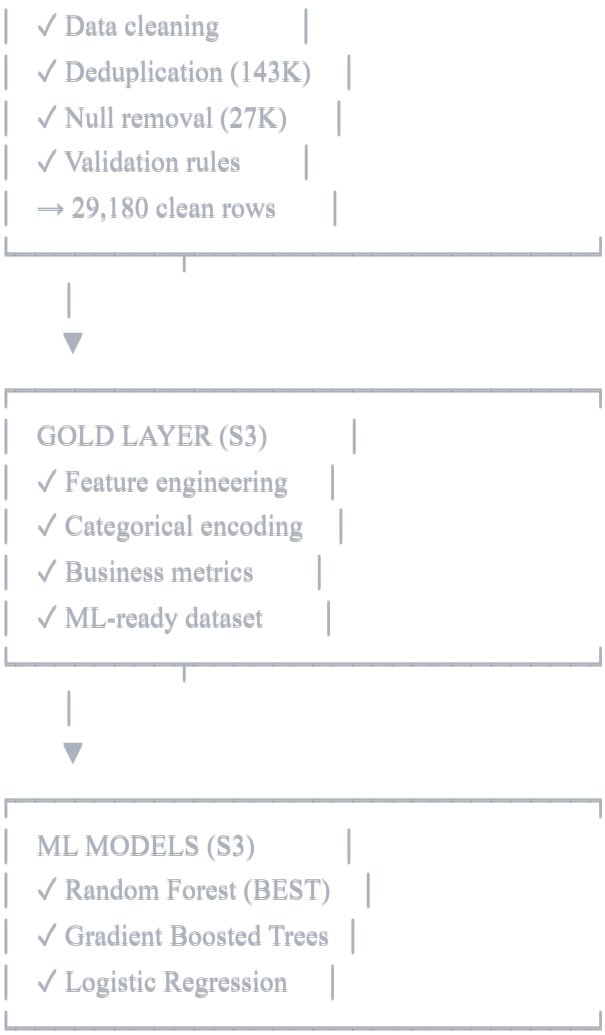
Problem Statement: Credit card companies lose significant revenue when customers churn. Early identification of at-risk customers enables proactive retention strategies.

Solution: Scalable ETL pipeline + ML model that identifies churners with 75.76% accuracy, enabling targeted retention campaigns.

🏗️ Architecture

Medallion Architecture (Bronze → Silver → Gold)





💻 Technology Stack

Cloud & Infrastructure

- **AWS S3**: Data lake storage (Bronze/Silver/Gold layers)
- **AWS IAM**: Secure access management
- **Apache Spark 3.5**: Distributed data processing
- **PySpark**: Python API for Spark

Machine Learning

- **Spark MLlib**: Scalable ML algorithms
- **Random Forest**: Best performing model (AUC: 0.80)
- **Gradient Boosted Trees**: Alternative ensemble method

- **Logistic Regression**: Baseline model

Development Tools

- **Python 3.10**: Core programming language
- **Java 17**: Required for Spark runtime
- **Hadoop 3.3.4**: Distributed file system support
- **Git**: Version control

✨ Key Features

Data Engineering

- ✅ **Scalable ETL Pipeline**: Processes 200K+ records efficiently
- ✅ **Data Quality**: 72% duplicate/null removal for clean analysis
- ✅ **Feature Engineering**: 6 derived features for ML
- ✅ **Parquet Format**: Columnar storage for optimized queries

Machine Learning

- ✅ **Multi-Model Training**: Compared 3 different algorithms
- ✅ **Cross-Validation**: 80/20 train-test split
- ✅ **Feature Importance**: Identified top churn drivers
- ✅ **Production Ready**: Serialized model saved to S3

Best Practices

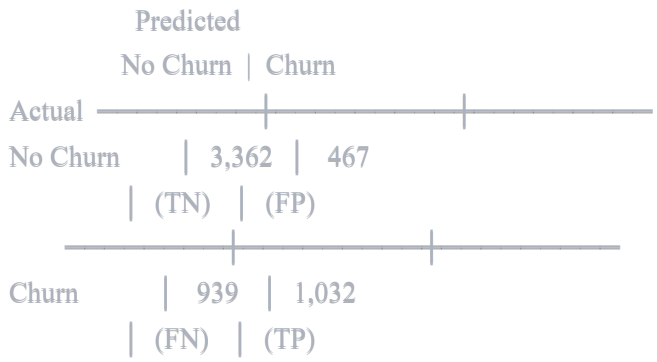
- ✅ **Medallion Architecture**: Industry-standard data layering
- ✅ **Cloud-Native**: Fully deployed on AWS infrastructure
- ✅ **Reproducible**: Automated pipeline scripts
- ✅ **Version Controlled**: All code in Git repository

📊 Results & Performance

Model Performance Comparison

Model	AUC-ROC	Accuracy	Precision	Recall	F1-Score
Random Forest ⭐	**0.8000**	**75.76%**	**75.00%**	**75.76%**	**0.7481**
Gradient Boosted Trees	0.7967	75.38%	74.58%	75.38%	0.7433
Logistic Regression	0.7812	74.10%	73.52%	74.10%	0.7185

Confusion Matrix (Random Forest)



Key Metrics:

- **True Positive Rate**: 52.4% (catches half of all churners)
- **False Positive Rate**: 12.2% (low - won't unnecessarily target loyal customers)
- **Precision for Churn**: 68.8% (when predicting churn, right 69% of the time)

Top 5 Churn Drivers

1. **Number of Accounts (31.5%)** - Customers with fewer accounts churn more
2. **Engagement Score (24.9%)** - Low engagement = high churn risk
3. **Customer Lifetime Value (15.2%)** - Lower CLV correlates with churn
4. **Transaction Frequency (9.0%)** - Infrequent users are at risk
5. **Customer Age (5.6%)** - Age demographic plays a role

🔄 Pipeline Stages

Stage 1: Bronze Layer (Raw Data Ingestion)

Script: `spark_bronze_to_silver.py`

- Read CSV from S3
- No transformations
- Store as-is in Parquet format
- **Input**: 199,999 rows
- **Output**: Raw data in S3 Bronze

Stage 2: Silver Layer (Data Cleaning)

Script: `spark_bronze_to_silver.py`

Transformations:

- Remove 143,978 duplicate records (72%)
- Drop 26,841 rows with null values
- Validate numeric ranges (age < 120, CLV ≥ 0, etc.)
- Standardize text columns (uppercase, trim)

Output: 29,180 clean records in S3 Silver

Stage 3: Gold Layer (Feature Engineering)

Script: `spark_silver_to_gold.py`

Created Features:

- `AGE_GROUP`: Young, Middle-Aged, Senior, Elderly
- `CLV_TIER`: Low, Medium, High, Very High
- `SALARY_BRACKET`: 5-tier salary classification
- `TRANSACTION_CATEGORY`: Rare, Regular, Frequent, Very Frequent
- `ENGAGEMENT_SCORE`: Composite metric (frequency + accounts)
- `HIGH_RISK_FLAG`: Binary flag for high-risk customers

Output: 29,180 rows with 12 features in S3 Gold

Stage 4: ML Training

Script: `train_churn_model.py`

Process:

1. Load Gold layer data from S3
2. Encode categorical features (StringIndexer)
3. Assemble feature vectors
4. Train 3 models with 80/20 split
5. Evaluate with multiple metrics
6. Save best model (Random Forest) to S3

Output: Trained model in S3 Models

🚀 Setup & Installation

Prerequisites

```bash
# Required software
- Python 3.10+
- Java 17 (for Spark)
- AWS CLI
- Git
```

Step 1: Clone Repository

```bash
git clone https://github.com/yourusername/credit-card-churn-prediction.git
cd credit-card-churn-prediction
```

Step 2: Create Virtual Environment

```bash
```

```
python -m venv spark_venv
spark_venv\Scripts\activate  # Windows
source spark_venv/bin/activate  # Linux/Mac
```

Step 3: Install Dependencies

bash

```
pip install pyspark==3.5.0
pip install boto3
pip install awscli
```

Step 4: Install Hadoop for Windows

bash

```
# Download winutils.exe and hadoop.dll
# Place in: C:\hadoop\bin\

# Set environment variable
$env:HADOOP_HOME = "C:\hadoop"
```

Step 5: Configure AWS Credentials

bash

```
aws configure
# Enter your AWS Access Key ID
# Enter your AWS Secret Access Key
# Default region: us-east-2
# Output format: json
```

Step 6: Create S3 Bucket

bash

```
aws s3 mb s3://your-bucket-name --region us-east-2

# Create folder structure
aws s3api put-object --bucket your-bucket-name --key bronze/
aws s3api put-object --bucket your-bucket-name --key silver/
aws s3api put-object --bucket your-bucket-name --key gold/
aws s3api put-object --bucket your-bucket-name --key models/
```

📖 Usage

1. Upload Raw Data to S3 Bronze

```
bash

aws s3 cp data/raw/cust_churn_train.csv s3://your-bucket-name/bronze/
```

2. Run Bronze → Silver Pipeline

```
bash

python scripts/spark_bronze_to_silver.py
```

3. Run Silver → Gold Pipeline

```
bash

python scripts/spark_silver_to_gold.py
```

4. Train ML Model

```
bash

python scripts/train_churn_model.py
```

5. Verify Results

```
bash
```

```
# Check all layers
aws s3 ls s3://your-bucket-name/bronze/
aws s3 ls s3://your-bucket-name/silver/
aws s3 ls s3://your-bucket-name/gold/
aws s3 ls s3://your-bucket-name/models/
```

💼 Business Impact

Key Insights

High-Risk Customer Segment:

- Identified 2,803 high-risk customers (9.6% of total)
- 45.4% churn rate among high-risk vs 34% overall
- Criteria: Low CLV (<$50K) + Low frequency (<5 transactions)

Churn Patterns by CLV:

- Medium CLV customers have **50.5% churn rate** (highest!)
- Low CLV customers: 47.1% churn rate
- Very High CLV customers: 24.5% churn rate (most loyal)

Age Demographics:

- Young customers (18-30): 44% churn rate
- Middle-aged (30-50): 34.2% churn rate
- Elderly (65+): 32.2% churn rate

Recommended Actions

1. **Immediate**: Target 2,803 high-risk customers with retention offers
2. **Strategic**: Focus on medium-CLV segment (highest churn rate)
3. **Proactive**: Increase engagement for low-frequency users
4. **Long-term**: Cross-sell additional accounts to single-account holders

ROI Potential

Assuming:

- Average customer value: $500/year
- Retention campaign cost: $50/customer
- Model prevents 25% of predicted churns (258 customers)

Estimated Annual Savings: 258 customers × $500 = **$129,000**
Campaign Cost: 1,032 targeted × $50 = **$51,600**
Net Benefit: **$77,400/year**

🔮 Future Enhancements

Technical Improvements

- [ ] Real-time prediction API with FastAPI
- [ ] Automated retraining pipeline (MLOps)
- [ ] A/B testing framework for retention strategies
- [ ] Dashboard for business stakeholders (Tableau/PowerBI)
- [ ] Model monitoring and drift detection

Feature Engineering

- [ ] Time-series features (monthly trends)
- [ ] Customer segmentation clustering
- [ ] Product usage patterns
- [ ] Social network analysis (referrals)

Model Optimization

- [ ] Hyperparameter tuning with GridSearch
- [ ] Ensemble stacking methods
- [ ] Deep learning models (Neural Networks)
- [ ] Explainable AI (SHAP values)

📁 Project Structure

```
credit-card-churn-prediction/
│
├── data/
│   ├── raw/                  # Original CSV files
│   └── processed/            # Local processed data
│
├── scripts/
│   ├── spark_bronze_to_silver.py   # ETL: Raw → Clean
│   ├── spark_silver_to_gold.py     # ETL: Clean → ML-ready
│   └── train_churn_model.py        # ML training pipeline
│
├── models/
│   └── saved_models/         # Local model checkpoints
│
├── notebooks/
│   └── exploratory_analysis.ipynb  # EDA notebooks
│
├── spark_venv/               # Python virtual environment
│
├── README.md                 # This file
├── requirements.txt          # Python dependencies
└── .gitignore
```

🎓 Skills Demonstrated

Data Engineering

- ETL pipeline design and implementation
- Data quality validation and cleansing
- Scalable data processing with Apache Spark
- Cloud data architecture (Medallion pattern)

Machine Learning

- Feature engineering and selection
- Multi-model comparison and evaluation

- Binary classification techniques
- Model interpretability (feature importance)

Cloud & DevOps

- AWS S3 data lake management
- IAM security configuration
- Distributed computing with Spark
- Production-ready code deployment

Business Acumen

- Customer churn analytics
- ROI calculation
- Strategic recommendations
- Stakeholder communication

📞 Contact

Your Name
📧 Email: your.email@example.com
💼 LinkedIn: linkedin.com/in/yourprofile
🐙 GitHub: github.com/yourusername
🌐 Portfolio: yourwebsite.com

📄 License

🙏 Acknowledgments

- Dataset: Credit Card Customer Churn Dataset
- Technology: Apache Spark, AWS
- Inspiration: Industry best practices in MLOps and data engineering

⭐ **If you found this project helpful, please star the repository!**